DOCUMENTACIÓN — Abraham Studio

Proyecto: Sitio web de Abraham Studio, barbería y estética masculina (Escazú, Costa Rica)
Tipo: Landing page estática de una sola página
Stack: Next.js 15 · React 19 · Tailwind CSS 3
Desarrollado por: Aurigital

Tabla de contenido

1. Documentación técnica del proyecto
2. Convenciones de código
3. Dependencias
4. Checklist de despliegue
5. Guía de Clean Code
6. Buenas prácticas de Handoff
7. Buenas prácticas de documentación técnica en Markdown
8. Deuda técnica y puntos de atención conocidos

---

1. Documentación técnica del proyecto

1.1 Descripción general

Abraham Studio es un sitio web de presentación y captación de citas para una barbería. Se compone de una única página (`/`) dividida en secciones navegables por ancla: presentación del barbero, servicios y precios, comunidad (horario, ubicación y medios), feed de Instagram, agendamiento en línea y pie de página.

Se construye como **sitio estático** (`output: 'export'` en `next.config.mjs`): `next build` genera HTML/CSS/JS en la carpeta `out/`, que puede servirse desde cualquier hosting estático. No hay backend propio, base de datos ni variables de entorno.

1.2 Arquitectura

Aspecto	Decisión
Framework	Next.js 15.1.4 con App Router (`app/`)
Renderizado	Exportación estática. Todos los componentes de la página se cargan con `next/dynamic` y `ssr: false`, por lo que el contenido se pinta en el cliente
Lenguaje	JavaScript (sin TypeScript). Alias `@/*` apuntando a la raíz definido en `jsconfig.json`
Estilos	Tailwind CSS 3 + CSS global (`app/globals.css`) con fuentes locales mediante `@font-face`
Animaciones	AOS (scroll), Framer Motion (hero), GSAP, react-just-parallax y react-scroll-parallax
Integraciones externas	Google Tag Manager, SimplyMeet (agenda), Elfsight (feed de Instagram), Google Maps (embed), Waze y WhatsApp (enlaces)

1.3 Estructura de carpetas

```
.
├── app/
│   ├── components/
│   │   ├── navbar.js              # Barra de navegación con anclas
│   │   ├── hero.js                # Portada con video y redes sociales
│   │   ├── AbrahamSection.js      # Presentación del barbero
│   │   ├── TreatmentsAndCare.js   # Servicios y precios (id="Servicios")
│   │   ├── comunidad.js           # Horario, mapa, medios y modal con iframe
│   │   ├── instagram.js           # Widget de Elfsight
│   │   ├── calendarhit.js         # Iframe de agendamiento SimplyMeet
│   │   └── footer.js              # Pie de página, contacto y créditos
│   ├── globals.css                # Tailwind, variables CSS y @font-face
│   ├── layout.js                  # Layout raíz, metadata SEO, GTM, JSON-LD
│   ├── page.js                    # Página principal y pantalla de carga
│   └── site.webmanifest
├── public/
│   ├── assets/                    # Imágenes (.avif/.png/.svg), videos (.webm)
│   │   └── fonts/                 # Copperplate y Futura (.ttf)
│   ├── HeroBG.png                 # Imagen Open Graph / schema.org
│   └── robots.txt
├── eslint.config.mjs
├── jsconfig.json
├── next.config.mjs
├── postcss.config.mjs
├── tailwind.config.mjs
└── package.json
```

1.4 Flujo de la página

1. `app/layout.js` define `<html lang="es">`, la metadata (título, descripción, Open Graph, canonical), el contenedor de Google Tag Manager, el script de Elfsight (`lazyOnload`) y el JSON-LD de tipo `BarberShop`.

2. `app/page.js` es un componente cliente. Muestra una pantalla de carga (`LogoLoading.svg`, fondo `#3C5954`) hasta que el evento `load` de la ventana se dispara y transcurren 2 segundos.
3. Pasada la carga se renderizan, en orden: `Navbar`, `Hero`, `AbrahamSection`, `TreatmentsAndCare`, `Comunidad`, `InstagramSection`, `Calendarhit`, `Footer`.

1.5 Secciones y anclas

Ancla	Componente	Contenido
`#Abraham`	`AbrahamSection`	Presentación y llamada a la acción hacia la agenda
`#Servicios`	`TreatmentsAndCare`	Corte de cabello, corte y barba, tinte y free style, membresía mensual, tratamiento capilar, entre otros, con precios en colones
`#Comunidad`	`Comunidad`	Mensaje de bienvenida, horario, mapa de Google, medios (modal con iframe) y redes sociales
`#Instagram`	`InstagramSection`	Widget Elfsight (`data-elfsight-app-lazy`)
`#Calendarhit`	`Calendarhit`	Iframe de `app.simplymeet.me/abrahamstudio`

1.6 Integraciones externas

Servicio	Dónde	Notas
Google Tag Manager	`app/layout.js`	ID `GTM-MBW74K45`, mediante `@next/third-parties/google`
SimplyMeet	`calendarhit.js`	Iframe de reservas. Cambios de servicios o disponibilidad se hacen en la plataforma, no en el código
Elfsight	`layout.js` e `instagram.js`	Script de plataforma + contenedor con el ID de la app de Instagram
Google Maps	`comunidad.js`	Iframe `embed` con la ubicación del estudio
Waze / WhatsApp	`footer.js`, `hero.js`, `comunidad.js`	Enlaces directos. WhatsApp: `+506 8424-9981`
Redes sociales	`hero.js`, `comunidad.js`, `footer.js`	Facebook e Instagram de Abraham

1.7 Identidad visual

- **Colores principales:** fondo `#131313`, rojo vino `#72140A` (navbar y tarjetas), verde `#3C5954` (carga), dorado `#D5A153` (hover).
- **Tipografías** (archivos locales en `public/assets/fonts`), expuestas como clases de Tailwind:

Clase Tailwind	Fuente
`font-cp-regular`	Copperplate Regular
`font-cp-bold`	Copperplate Bold
`font-futura-bold`	Futura Bold
`font-futura-medium`	Futura Medium
`font-futura-medium-condensed`	Futura Medium Condensed

1.8 Puesta en marcha local

Requisitos: Node.js 18.18 o superior (recomendado LTS) y npm.

```
npm install        # instala dependencias
npm run dev        # servidor de desarrollo (Turbopack) en http://localhost:3000
npm run lint       # análisis estático con ESLint
npm run build      # genera el sitio estático en out/
```

Script	Comando	Uso
`dev`	`next dev --turbopack`	Desarrollo local con recarga en caliente
`build`	`next build`	Compilación de producción y exportación estática
`start`	`next start`	No aplica a una exportación estática; usar un servidor de archivos sobre `out/`
`lint`	`next lint`	Reglas `next/core-web-vitals`

Para probar la exportación localmente: `npx serve out`.

1.9 Tareas de mantenimiento frecuentes

Necesidad	Qué modificar
Cambiar precios o servicios	`app/components/TreatmentsAndCare.js`
Cambiar horario, teléfono o dirección	`footer.js`, `comunidad.js` y el JSON-LD de `layout.js` (mantenerlos sincronizados)
Cambiar textos SEO	`metadata` en `app/layout.js`
Reemplazar videos o imágenes	`public/assets/` (conservar el mismo nombre o actualizar las referencias)
Cambiar el ID de GTM	`<GoogleTagManager gtmId="..." />` en `app/layout.js`
Cambiar el widget de Instagram	Atributo `elfsight-app-<id>` en `instagram.js`
Cambiar el calendario de reservas	`src` del iframe en `calendarhit.js`

---

2. Convenciones de código

2.1 Nombrado

Elemento	Convención	Ejemplo
Componentes React	`PascalCase`, tanto la función como el archivo	`Navbar`, `TreatmentsAndCare`
Funciones y variables	`camelCase`, con verbos para acciones	`openModalWithUrl`, `isModalOpen`
Constantes	`UPPER_SNAKE_CASE` si son globales e inmutables	`CALENDAR_URL`
Estados booleanos	Prefijo `is`, `has`, `should`	`isModalOpen`
Anclas (`id`)	`PascalCase`, iguales a las usadas en `href="#..."`	`Comunidad`, `Servicios`
Assets	Sin espacios ni tildes en archivos nuevos; nombres descriptivos	`hero-video.webm`
Commits	Conventional Commits en español o inglés, de forma consistente	`feat: agrega sección de membresías`

> Los componentes existentes `navbar.js`, `calendarhit.js`, `comunidad.js`, `footer.js`, `hero.js` e `instagram.js` están en minúscula; los archivos nuevos y cualquier renombrado futuro deben seguir `PascalCase`. Ver sección 8.

2.2 Estructura de un componente

1. Directiva `"use client"` solo si el componente usa estado, efectos o APIs del navegador.
2. Imports en este orden: React/Next → librerías de terceros → módulos locales → estilos.
3. Constantes y datos estáticos fuera del componente.
4. Hooks al inicio de la función, sin condicionales.
5. Funciones auxiliares antes del `return`.
6. JSX limpio; extraer a subcomponentes lo que se repita.
7. `export default` al final o en la declaración, de forma uniforme dentro del proyecto.

```
"use client";
import { useState } from "react";
import Link from "next/link";

const WHATSAPP_URL = "https://api.whatsapp.com/send?phone=50684249981";

export default function ContactButton({ label }) {
  const [isHovered, setIsHovered] = useState(false);

  return (
    <Link
      href={WHATSAPP_URL}
      target="_blank"
      rel="noopener noreferrer"
      onMouseEnter={() => setIsHovered(true)}
      onMouseLeave={() => setIsHovered(false)}
    >
      {label}
    </Link>
  );
}
```

2.3 Estilos con Tailwind

- Usar utilidades de Tailwind; reservar `globals.css` para variables, `@font-face` y reglas globales.
- Orden sugerido de clases: layout → tamaño y espaciado → tipografía → color → efectos → estados → responsive. Mobile first (`md:`, `lg:` para pantallas mayores).
- Los colores y fuentes de marca se definen una sola vez en `tailwind.config.mjs` y se consumen por nombre; evitar repetir valores hexadecimales arbitrarios (`bg-[#72140A]`) cuando exista o pueda crearse un token.
- Las fuentes personalizadas se usan por clase (`font-cp-bold`), nunca con `style` en línea.
- Evitar `style={{ }}` salvo para valores dinámicos que Tailwind no pueda expresar.

2.4 Imágenes, video y rendimiento

- Preferir `next/image` con `alt` descriptivo y `sizes` correctos. Usar `priority` solo en imágenes visibles al cargar (logo, hero).
- Formatos: imágenes en `.avif`, video en `.webm`, logos en `.svg`.
- Todo `<img>` o `<video>` debe tener texto alternativo o equivalente accesible, y los videos decorativos `muted`, `loop` y `playsInline`.
- Optimizar el peso de cada asset antes de subirlo al repositorio.

2.5 Enlaces y seguridad

- Todo enlace externo con `target="_blank"` lleva `rel="noopener noreferrer"`.
- Todo `<iframe>` lleva `title` descriptivo y, si es posible, `loading="lazy"`.
- No incluir tokens, claves ni credenciales en el código. Los archivos `.env*` están excluidos por `.gitignore`.
- `dangerouslySetInnerHTML` solo se admite para JSON-LD generado con `JSON.stringify` desde datos propios.

2.6 Formato y linting

- Indentación de 2 espacios, comillas dobles, punto y coma al final de cada sentencia.
- Antes de cada commit: `npm run lint` sin errores ni advertencias nuevas.
- Sin código comentado ni `console.log` en commits finales.

2.7 Flujo de Git

- La rama `main` contiene siempre código listo para producción.
- El trabajo se hace en ramas de funcionalidad con prefijo: `feat/`, `fix/`, `docs/`, `chore/`.
- Commits pequeños, atómicos y con mensaje descriptivo en tiempo presente.
- Los cambios llegan a `main` mediante Pull Request con al menos una revisión.

---

3. Dependencias

3.1 Dependencias de producción

Paquete	Versión	Propósito
`next`	15.1.4	Framework principal (App Router, exportación estática)
`react` / `react-dom`	^19.0.0	Librería de interfaz
`@next/third-parties`	^16.3.0	Integración de Google Tag Manager
`aos`	^2.3.4	Animaciones al hacer scroll (`data-aos`)
`framer-motion`	^11.18.0	Animaciones de entrada del hero
`gsap`	^3.12.5	Animaciones avanzadas
`imagehover.css`	^2.0.0	Efectos hover sobre imágenes
`lucide-react`	^0.471.1	Iconos
`react-icons`	^5.4.0	Iconos de redes sociales (Font Awesome, Ant Design)
`react-just-parallax`	^3.1.16	Efecto parallax del hero
`react-scroll-parallax`	^3.4.5	Parallax en la sección de comunidad
`react-slick` / `slick-carousel`	^0.30.3 / ^1.8.1	Carruseles

3.2 Dependencias de desarrollo

Paquete	Versión	Propósito
`eslint`	^9	Análisis estático
`eslint-config-next`	15.1.4	Reglas de Next.js (`core-web-vitals`)
`@eslint/eslintrc`	^3	Compatibilidad flat config
`tailwindcss`	^3.4.1	Framework de utilidades CSS
`postcss`	^8	Procesador CSS requerido por Tailwind

3.3 Servicios de terceros

Servicio	Tipo	Responsable de la cuenta
Google Tag Manager (`GTM-MBW74K45`)	Analítica	Definir en el handoff
SimplyMeet (`abrahamstudio`)	Reservas	Cliente
Elfsight	Widget de Instagram	Definir en el handoff
Google Maps	Mapa embebido	—
Hosting	Sitio estático	Definir en el handoff

3.4 Política de actualización

- Mantener `package-lock.json` versionado; usar `npm ci` en entornos de integración y despliegue.
- Antes de actualizar versiones mayores (Next, React, Tailwind), crear una rama aparte, ejecutar `npm run build` y revisar visualmente cada sección.
- `eslint-config-next` debe mantenerse en la misma versión que `next`.
- Ejecutar `npm audit` periódicamente y atender vulnerabilidades altas o críticas.
- Retirar las dependencias que no se utilicen (ver sección 8).

---

4. Checklist de despliegue

4.1 Antes de desplegar

- ☐ Estoy en `main` actualizada (`git pull`) y sin cambios locales sin commitear.
- ☐ `npm ci` se ejecuta sin errores.
- ☐ `npm run lint` pasa sin errores.
- ☐ `npm run build` termina correctamente y genera la carpeta `out/`.
- ☐ Revisé el sitio exportado localmente (`npx serve out`).
- ☐ No hay `console.log`, código comentado ni credenciales en el código.
- ☐ Los datos de negocio están actualizados: precios, horario, teléfono y dirección (también en el JSON-LD de `layout.js`).

4.2 Verificación funcional

- ☐ La pantalla de carga aparece y da paso al contenido.
- ☐ Todos los enlaces del navbar y del footer llevan a su ancla.
- ☐ Los botones "Agenda" llevan a `#Calendarhit` y el iframe de SimplyMeet carga.
- ☐ El mapa de Google, el feed de Instagram y el modal de medios funcionan.
- ☐ Los enlaces de WhatsApp, Facebook, Instagram y Waze abren el destino correcto.
- ☐ Los videos reproducen (autoplay silenciado) y las imágenes cargan.
- ☐ Las tipografías de marca se ven correctamente.

4.3 Responsive y navegadores

- ☐ Móvil (≈375 px), tableta (≈768 px) y escritorio (≥1280 px).
- ☐ Chrome, Safari (incluido iOS), Firefox y Edge.
- ☐ Sin scroll horizontal ni elementos cortados.

4.4 SEO, rendimiento y accesibilidad

- ☐ `title`, `description`, Open Graph y canonical correctos en `layout.js`.
- ☐ La imagen `HeroBG.png` existe en `public/` y se previsualiza bien al compartir el enlace.
- ☐ `robots.txt` accesible en la raíz del dominio.
- ☐ JSON-LD válido (probar con la herramienta de resultados enriquecidos de Google).
- ☐ Lighthouse en móvil: Rendimiento, Accesibilidad, Buenas prácticas y SEO revisados; sin regresiones frente a la versión anterior.
- ☐ Todas las imágenes tienen `alt`.

4.5 Analítica

- ☐ GTM carga con el ID `GTM-MBW74K45` (verificar con Tag Assistant o el modo vista previa).
- ☐ Las etiquetas y eventos esperados se disparan.

4.6 Despliegue

- ☐ Publicar el contenido de `out/` en el hosting estático (el directorio completo, no solo `index.html`).
- ☐ Dominio y certificado HTTPS vigentes.
- ☐ Caché/CDN purgado si aplica.
- ☐ Crear una etiqueta de versión en Git (`git tag vX.Y.Z`) y subirla.

4.7 Después de desplegar

- ☐ Abrir la URL de producción en modo incógnito y repetir la verificación funcional.
- ☐ Comprobar que la consola del navegador no muestra errores.
- ☐ Confirmar que el flujo completo de reserva funciona.
- ☐ Avisar al cliente y al equipo con el enlace y la versión publicada.

4.8 Plan de reversa

- ☐ Conservar la compilación anterior (`out/` previo o etiqueta de Git).
- ☐ Si algo falla: volver a la etiqueta anterior, ejecutar `npm ci && npm run build` y volver a publicar.
- ☐ Registrar el incidente y su causa en el historial del proyecto.

---

5. Guía de Clean Code

5.1 Principios

1. **Nombres que revelan intención.** El nombre debe decir qué es o qué hace, sin necesidad de comentario. `isModalOpen` en lugar de `flag`.
2. **Funciones y componentes pequeños, con una sola responsabilidad.** Si un componente supera aproximadamente 150 líneas o mezcla varias secciones, dividirlo.
3. **DRY (no repetir).** Si el mismo bloque JSX o los mismos valores se repiten tres veces, extraer un subcomponente o una constante.
4. **KISS.** Elegir la solución más simple que cumpla el requisito.
5. **YAGNI.** No añadir librerías ni código "por si acaso".
6. **Sin números ni textos mágicos.** Valores como la URL de WhatsApp, los precios o los tiempos de animación se declaran como constantes con nombre.
7. **Comentarios para el porqué, no para el qué.** Si el código necesita explicar qué hace, mejorar el código.
8. **Dejar el código mejor de como se encontró** (regla del boy scout), sin mezclar refactors grandes con cambios funcionales.

5.2 Aplicación en este proyecto

Separar datos de presentación. Los servicios y precios hoy están escritos dentro del JSX. Lo recomendable es definirlos como datos y renderizarlos con `map`:

```
// Antes: bloque repetido por cada servicio
<div className="group ...">
  <h3>CORTE DE CABELLO</h3>
  <p>₡12,000</p>
  ...
</div>

// Después: datos + un solo componente
const SERVICES = [
  { id: "corte", title: "CORTE DE CABELLO", price: "₡12,000", media: "/assets/ServiceImage1.avif" },
  { id: "corte-barba", title: "CORTE Y BARBA", price: "₡18,000", media: "/assets/service3.webm" },
];

{SERVICES.map((service) => (
  <ServiceCard key={service.id} {...service} />
))}
```

Centralizar constantes de contacto. Teléfono, enlaces de redes, horario y dirección aparecen en varios archivos. Crear un módulo único, por ejemplo `app/data/business.js`, y consumirlo desde `hero`, `comunidad`, `footer` y el JSON-LD.

Evitar efectos duplicados. `AOS.init()` se invoca en `navbar.js` y en `hero.js`. Inicializar una sola vez en un único punto.

Componentes con nombre claro. Un componente es `PascalCase` (`Comunidad`, no `comunidad`) para que React y el linter lo traten como tal.

Limpiar imports. Eliminar los no usados (por ejemplo, íconos o utilidades importados que no se renderizan).

5.3 Señales de alerta (code smells)

Señal	Qué hacer
Componente de más de 150–200 líneas	Dividir en subcomponentes
JSX copiado y pegado	Extraer componente y usar `map`
Clases de Tailwind extremadamente largas repetidas	Extraer componente o token de diseño
Valores hexadecimales o URLs repetidos	Constante o token en `tailwind.config.mjs`
Imports sin usar	Eliminar
Código comentado	Eliminar; Git conserva el historial
Más de un nivel de condicionales anidados en JSX	Retornos tempranos o variables intermedias
`useEffect` que hace varias cosas	Un efecto por responsabilidad

5.4 Checklist de revisión de código

- ☐ ¿Los nombres se entienden sin contexto?
- ☐ ¿Cada componente hace una sola cosa?
- ☐ ¿Hay duplicación que se pueda extraer?
- ☐ ¿Se eliminaron `console.log`, código comentado e imports sin usar?
- ☐ ¿Los enlaces externos tienen `rel="noopener noreferrer"` y los iframes `title`?
- ☐ ¿Las imágenes tienen `alt` y los assets están optimizados?
- ☐ ¿Se verificó en móvil y escritorio?
- ☐ ¿`npm run lint` y `npm run build` pasan?

---

6. Buenas prácticas de Handoff

El handoff es la transferencia ordenada del proyecto a otra persona o equipo (cliente, mantenimiento o nuevo desarrollador). El objetivo es que quien lo reciba pueda operar y evolucionar el sitio sin depender de quien lo construyó.

6.1 Qué debe incluir una entrega

Entregable	Detalle
Repositorio	Acceso al repositorio, rama `main` estable y etiqueta de versión entregada
Documentación	Este documento actualizado, más el `README.md` con instrucciones de arranque
Accesos y cuentas	Lista de cuentas y propietarios: hosting, dominio y DNS, GTM, SimplyMeet, Elfsight y redes sociales
Credenciales	Entregadas por un gestor de contraseñas o canal seguro; **nunca** por correo, chat o dentro del repositorio
Assets originales	Logotipos, tipografías con su licencia, videos e imágenes fuente en máxima calidad
Diseño	Enlace al archivo de diseño (Figma u otro) y guía de marca
Proceso de despliegue	Checklist de la sección 4 y dónde se publica
Pendientes conocidos	Deuda técnica y mejoras sugeridas (sección 8)

6.2 Plantilla de acta de entrega

```
## Acta de entrega — Abraham Studio

- Fecha:
- Versión entregada (tag):
- Entrega: (nombre / rol)
- Recibe: (nombre / rol)

### Accesos transferidos
| Sistema | Propietario actual | Método de entrega | Confirmado |
| --- | --- | --- | --- |
| Repositorio | | | [ ] |
| Hosting | | | [ ] |
| Dominio / DNS | | | [ ] |
| Google Tag Manager | | | [ ] |
| SimplyMeet | | | [ ] |
| Elfsight | | | [ ] |

### Estado del proyecto
- Funcionalidades completadas:
- Pendientes / bugs conocidos:
- Riesgos:

### Contacto de soporte posterior
- Periodo de garantía:
- Canal y responsable:
```

6.3 Buenas prácticas

1. **Documentar mientras se construye**, no al final. Cada decisión no obvia debe quedar escrita (por qué se eligió algo, no solo qué).

2. **Dejar el proyecto en estado verde:** `npm ci`, `npm run lint` y `npm run build` funcionan en una máquina limpia. Probarlo antes de entregar.
3. **Reunión de traspaso** (30–60 min) con demostración: levantar el proyecto, hacer un cambio de precio, compilar y desplegar.
4. **Registrar decisiones y limitaciones** (por ejemplo, que el sitio es estático y que SimplyMeet gestiona la agenda).
5. **Entregar la propiedad de las cuentas al cliente**, no solo accesos temporales. Evitar cuentas personales de quien desarrolla.
6. **Documentar los puntos de cambio frecuentes** (ver 1.9) para que el cliente o el mantenedor no necesiten leer todo el código.
7. **Limpiar antes de entregar:** ramas obsoletas, archivos temporales, dependencias sin usar y secretos en el historial.
8. **Definir el periodo de soporte** y el canal de contacto después de la entrega.
9. **Confirmar la recepción por escrito**: el receptor valida que pudo ejecutar el proyecto y acceder a todas las cuentas.

6.4 Checklist de handoff

- ☐ Repositorio limpio, `main` estable y versión etiquetada.
- ☐ `README.md` y esta documentación vigentes.
- ☐ El receptor ejecutó con éxito `npm ci`, `npm run dev` y `npm run build`.
- ☐ Todos los accesos transferidos y confirmados; credenciales entregadas de forma segura.
- ☐ Assets originales y archivos de diseño entregados.
- ☐ Reunión de traspaso realizada y minuta compartida.
- ☐ Pendientes y deuda técnica documentados.
- ☐ Periodo y canal de soporte acordados.
- ☐ Acta de entrega firmada o aceptada por escrito.

---

7. Buenas prácticas de documentación técnica en Markdown

7.1 Principios

- **Escribir para quien llega por primera vez.** Asumir que el lector no conoce el proyecto.
- **Una fuente de verdad.** No duplicar información; enlazar a la sección correspondiente.
- **Actualizar con el código.** Todo cambio que afecte comandos, estructura, dependencias o despliegue incluye su actualización de documentación en el mismo Pull Request.
- **Breve, concreta y verificable.** Cada comando documentado debe haberse probado.

7.2 Estructura recomendada

1. Título y descripción corta.
2. Tabla de contenido (en documentos largos).
3. Requisitos y puesta en marcha.
4. Arquitectura y estructura de carpetas.
5. Convenciones y flujo de trabajo.
6. Despliegue y operación.
7. Problemas conocidos y contacto.

7.3 Reglas de estilo en Markdown

Elemento	Regla
Títulos	Un único `#` por documento; jerarquía sin saltos (`##` → `###`)
Bloques de código	Siempre con lenguaje (```` ```bash ````, ```` ```jsx ````, ```` ```text ````)
Comandos y archivos	En `código en línea`: `npm run build`, `app/layout.js`
Listas	`-` para no ordenadas; `1.` para pasos secuenciales
Tablas	Para comparar datos; encabezados claros y celdas cortas
Enlaces	Texto descriptivo (no "clic aquí"); preferir rutas relativas dentro del repositorio
Checklists	`- [ ]` para tareas verificables
Notas	Citas `>` para advertencias o aclaraciones
Líneas	Un párrafo por línea lógica; evitar líneas de más de ~120 caracteres si no son tablas
Imágenes	Con texto alternativo; guardadas en una carpeta del repositorio
Idioma	Un solo idioma por documento y términos técnicos coherentes

7.4 Qué documentar y dónde

Contenido	Ubicación
Arranque rápido	`README.md`
Documentación técnica completa	`DOCUMENTACIÓN_Abraham_Studio.md`
Decisiones de arquitectura	Sección dedicada o carpeta `docs/decisions/` (ADR)
Cambios entre versiones	`CHANGELOG.md`
Cómo contribuir	`CONTRIBUTING.md`

7.5 Mantenimiento de la documentación

- Revisar la documentación en cada entrega o versión mayor.
- Incluir la fecha de la última revisión y la versión del proyecto documentada.
- Pedir a una persona ajena al proyecto que siga la guía de arranque; todo lo que no logre hacer indica un hueco.
- Formatos derivados (Word y PDF) se generan siempre desde el archivo `.md`, que es la fuente de verdad. No editar los derivados a mano.

---

8. Deuda técnica y puntos de atención conocidos

Observaciones detectadas al revisar el código, ordenadas por prioridad sugerida. No son errores bloqueantes, pero conviene atenderlas.

#	Hallazgo	Impacto	Recomendación
1	`@next/third-parties` está en `^16.3.0` mientras `next` es `15.1.4`	Posibles incompatibilidades entre versiones	Alinear la versión de `@next/third-parties` con la de Next
2	Todos los componentes se cargan con `ssr: false` y la página muestra una carga de 2 s	El HTML exportado queda casi vacío, lo que perjudica SEO y tiempo de carga percibido	Renderizar el contenido en el servidor/estático y mantener solo en cliente lo que requiera el navegador; reducir o eliminar el retardo artificial
3	`site.webmanifest` tiene `name` y `short_name` vacíos y referencia `android-chrome-192x192.png`, que no existe	Manifiesto inválido; error 404	Completar el manifiesto o eliminarlo, y añadir los íconos
4	`images.formats` y `minimumCacheTTL` no tienen efecto con `output: 'export'` (no hay optimizador de imágenes en tiempo de ejecución)	Configuración engañosa	Usar `images: { unoptimized: true }` y optimizar los archivos de forma manual
5	Nombres de archivo y componentes inconsistentes (`navbar.js`, `comunidad.js`, `calendarhit.js` en minúscula; `comunidad` como función de componente)	Incumple la convención de React y dificulta el linter	Renombrar a `PascalCase`
6	`AOS.init()` se invoca en `navbar.js` y `hero.js`	Inicialización duplicada	Centralizar en un único lugar
7	Servicios, precios, horario, teléfono y redes están repetidos y escritos en el JSX	Riesgo de inconsistencias al actualizar	Extraer a un módulo de datos (ver 5.2)
8	Se usan etiquetas `<img>` en `calendarhit.js`; el iframe de SimplyMeet no tiene `title` ni `loading="lazy"` y el del mapa no tiene `title`	Rendimiento y accesibilidad	Usar `next/image` y completar atributos
9	Dependencias sin uso en `app/` (verificado por búsqueda): `gsap`, `lucide-react`, `react-slick`, `slick-carousel`	Peso innecesario del bundle	Retirarlas con `npm uninstall`
10	Archivos de fuente con espacios en el nombre (`Copperplate Regular.ttf`, `Futura Bold font.ttf`)	Frágil en algunos servidores/URLs	Renombrar sin espacios y actualizar `globals.css`
11	Hay `Geist` y `Geist_Mono` cargadas desde Google Fonts pero el diseño usa Copperplate/Futura	Peticiones y peso extra	Retirarlas si no se usan
12	El `README.md` es la plantilla por defecto de `create-next-app`	Información poco útil para nuevos integrantes	Sustituirlo por un resumen del proyecto que enlace a esta documentación

---

Documento mantenido junto al código. Actualizar al cambiar comandos, estructura, dependencias o proceso de despliegue.